



Introduzione a Python

Advanced Computer Programming

Prof. Raffaele Della Corte



Sommario

- Introduzione a Python
- Oggetti, Stringhe
- Flusso di controllo: If then else, While, For
- Funzioni
- Tuple, List, Dizionari

Riferimenti

- Tony Gaddis. **Introduzione a Python**. 5° ed. – Pearson, 2021
- Paul J. Deitel, Harvey M. Deitel. **Introduzione a Python. Per l'informatica e la data science**. Pearson, 2021
- **Python: How to Think Like a Computer Scientist interactive edition**
<https://runestone.academy/runestone/books/published/thinkcspy/index.html>
- Allen Downey. **Think Python** - <https://greenteapress.com/thinkpython2/thinkpython2.pdf>



Compilatori vs Interpreti

- Programmi scritti in linguaggio di alto livello devono essere tradotti in linguaggio macchina per essere eseguiti
- **Compilatori:** **traducono** programmi scritti in linguaggi di alto livello in un programma separato, scritto in linguaggio macchina
 - Un programma scritto in linguaggio macchina può essere eseguito in qualunque altro momento
- **Interpreti:** **traducono ed eseguono** le istruzioni di un programma scritto in linguaggio di alto livello
 - **Utilizzato dal linguaggio Python**
 - Interpreta una istruzione alla volta
 - Non c'è un programma separato scritto in linguaggio macchina



Interprete Python (dettagli)

- L'implementazione di riferimento dell'interprete Python è **CPython**
 - CPython è scritto sia in C che in Python stesso
- Non converte il codice sorgente in istruzioni in linguaggio macchina ma *trasforma l'intero codice in qualcosa chiamato **byte code***
 - All'interno dell'interprete Python **avviene comunque una compilazione**, ma tale compilazione non converte l'intero codice in linguaggio macchina o assembly come avviene in linguaggi compilati come C/C++
 - **byte code e istruzioni assembly non sono la stessa cosa**
 - Il **byte code** viene generato all'interno di una *virtual machine* e per una *virtual machine* (*software di sistema intermedio*)
 - Il **linguaggio assembly** viene creato per una specifica CPU



Interprete Python (dettagli)

■ Il compilatore Python

- Legge le istruzioni di un programma scritto in linguaggio di alto livello o il codice sorgente di Python
- Verifica se le istruzioni sono formattate correttamente, cioè verifica la struttura sintattica di ogni riga del programma
- Nel caso in cui si riscontri un errore, la traduzione viene immediatamente interrotta e viene visualizzato un messaggio di errore
- Se non ci sono errori, il compilatore traduce le istruzioni di alto livello nel linguaggio intermedio equivalente che è il **byte code**

■ Il *byte code* viene fornito al vero è proprio interprete Python, la cosiddetta **Python Virtual Machine (PVM)**

- La **PVM** converte il *byte code* Python in istruzioni a livello macchina o in codice binario equivalente
- Se si verifica un errore in questa fase di interpretazione, la conversione si arresta mostrando un messaggio di errore



Compilatori vs Interpreti

- Python deve essere installato e configurato prima dell'uso
 - Installare l'interprete Python
- L'interprete Python può essere utilizzato in due modalità:
 - **Interactive mode**: scrivere le istruzioni Python ed eseguirle una alla volta
 - **Script mode**: scrivere tutte le istruzioni e salvarle in uno script Python da eseguire in un secondo momento



Interactive mode

- Quando eseguiamo Python in *interactive mode*, utilizziamo un **prompt dei comandi**
 - L'interprete rimane in attesa di istruzioni Python da scrivere
 - Il prompt dei comandi riappare dopo l'esecuzione dell'istruzione precedente
 - Se l'istruzione è non corretta saranno mostrati eventuali messaggi di errore
- Questa modalità è un buon modo per iniziare a capire Python



Script mode

- Dobbiamo utilizzare la **script mode** per eseguire un intero programma Python
 - Salvare l'insieme delle istruzioni Python in un file con estensione .py (raccomandato)
 - Eseguire da terminale il file Python invocando l'interprete
 - `python filename.py`
 - L'interprete Python valuterà passo passo tutte le istruzioni scritte nello script
- In questa modalità vengono forniti i classici meccanismi per ottenere **argomenti da linea di comando** e/o **redirezione di I/O**
- Python possiede anche un meccanismo per consentire ai programmi Python di **agire sia da script** che da **modulo** da importare ed essere usato da altri programmi Python



Byte code compilato: i file .pyc

- I **file .pyc** sono file **byte code compilati** che vengono generati dall'interprete Python quando uno **script Python viene importato o eseguito**
- I file .pyc possono essere **eseguiti direttamente dall'interprete**, senza la necessità di ricompilare il codice sorgente a ogni esecuzione dello script
 - Tempi di esecuzione degli script più rapidi
- Per ottenere direttamente il file .pyc a partire da un file .py eseguire
 - `python -m compileall filename.py`



Programmi Python

- Un **programma Python** è una sequenza di definizioni e comandi
 - Le definizioni sono **valutate**
 - I comandi sono **eseguiti** dall'interprete Python in una shell (terminal)
- **I comandi** (statements) istruiscono l'interprete a fare qualcosa
- Tali comandi possono essere digitati direttamente all'interno di una **shell** oppure immagazzinati in un **file** che viene letto dalla shell per essere valutato



Python IDEs

- **VSCodium** (installazione estensioni)
- **Anaconda Navigator**
<https://www.anaconda.com/products/individual>
 - Notebook Jupiter
- **PyCharm**
<https://www.jetbrains.com/pycharm/download/>
- **Terminale...**



Estensioni VSCodium

- **Estensioni da installare**
 - Code Runner (formulahendry)
 - Jupyter (ms-toolsai)
 - Jupyter Cell Tags (ms-toolsai)
 - Jupyter Keymap (ms-toolsai)
 - Jupyter Notebook Renderers (ms-toolsai)
 - Jupyter Slide Show (ms-toolsai)
 - Python (ms-python)
 - Python Debugger (ms-python)
- N.B.: Le estensioni non sostituiscono l'interprete Python, pertanto è necessario installarlo sul pc



Oggetti Python

- I programmi Python manipolano i cosiddetti **data objects**
- Gli oggetti hanno un **tipo** che definisce il genere di cose che il programma può fare su di essi
- Gli oggetti possono essere:
 - **scalari** (non possono essere divisi)
 - **non-scalari** (hanno una struttura interna che può essere acceduta)



Oggetti scalari

- `int` – rappresenta **interi**, e.g., 5
- `float` – rappresenta **numeri reali**, e.g., 3.27
- `bool` – rappresenta i valori **booleani** `True` e `False`
- `NoneType` – tipo **speciale** e assume un solo valore, `None`
- Possiamo usare `type()` per vedere il tipo di un oggetto

```
>>> type(5)
```

```
int
```

```
>>> type(3.0)
```

```
float
```

*what you write into
the Python shell*

*what shows after
hitting enter*



Conversione di tipo (*type casting*)

- Possiamo **convertire un oggetto di un tipo in un oggetto di altro tipo**
- `float(3)` converte l'intero 3 in un float 3.0
- `int(3.9)` tronca il float 3.9 nell'intero 3



Stampa verso la console

- Per poter mostrare l'output dal codice verso l'utente, possiamo usare il comando `print`

```
In [11]: 3+2
```

```
Out[11]: 5
```

*"Out" tells you it's an
interaction within the
shell only*

```
In [12]: print(3+2)
```

```
5
```

*No "Out" means it is
actually shown to a user,
apparent when you
edit/run files*



Operazioni per interi e float

- $i + j$ → la **somma**
 - $i - j$ → la **differenza**
 - $i * j$ → il **prodotto**
 - i / j → **divisione**
- Se entrambi sono interi, il risultato sarà un intero
 - Se entrambi sono float, il risultato sarà un float
- Il risultato è un float

- $i \% j$ → l'operazione di **modulo** quando i viene diviso per j
- $i ** j$ → **elevamento a potenza** di i a j



Stringhe

- Possono contenere lettere, caratteri speciali, spazi, numeri, etc.
- Sono definite utilizzando **doppio apice (*quotation marks*) o singolo apice (*single quotes*)**
 - `hi = "hello there"`
 - `hi2 = 'hello there again'`
- **E' possibile Concatenare** stringhe
 - `name = "pippo"`
 - `greet = hi + name`
 - `greeting = hi + " " + name`
- Eseguire **operazioni** su stringhe come definito nella documentazione Python
 - `silly = hi + " " + name * 3`



Stringhe

- Possiamo pensare alle stringhe come una **sequenza** case-sensitive di caratteri
- Possiamo comparare stringhe attraverso gli operatori `==`, `>`, `<`, etc.
- `len()` è una funzione utilizzata per ottenere la **lunghezza** di una stringa passata come primo argomento

```
s = "abc"
```

```
len(s) → ritorna 3
```



Stringhe

- Le parentesi quadre sono usate per **indirizzare** una stringa e accedere ad un suo valore ad una certa posizione/indice

```
s = "abc"
```

index: 0 1 2 ← indexing always starts at 0

index: -3 -2 -1 ← last element always at index -1

s[0] → ritorna "a"

s[1] → ritorna "b"

s[2] → ritorna "c"

s[3] → trying to index out of bounds, error

s[-1] → ritorna "c"

s[-2] → ritorna "b"

s[-3] → ritorna "a"



Stringhe

- Possiamo affettare (**slice**) le stringhe utilizzando la notazione `[start:stop:step]`
- Con `[start:stop]`, `step=1` by default
- Possiamo anche utilizzare solo la notazione `::`

```
s = "abcdefgh"
```

```
s[3:6] → ritorna "def", same as s[3:6:1]
```

```
s[3:6:2] → ritorna "df"
```

```
s[:] → ritorna "abcdefgh", same as s[0:len(s):1]
```

```
s[::-1] → ritorna "hgfedcba", same as s[-1:-len(s):-1]
```

```
s[4:1:-2] → ritorna "ec"
```

*If unsure what some
command does, try it
out in your console!*



Stringhe

- Le stringhe sono “**immutabili**” – non possono essere modificate

```
s = "hello"
```

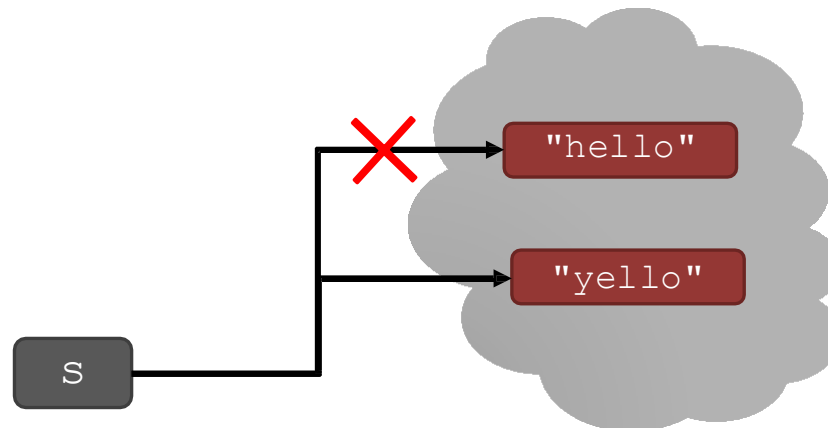
```
s[0] = 'y'
```

→ gives an error

```
s = 'y'+s[1:len(s)]
```

→ is allowed,

s bound to new object





INPUT/OUTPUT: print

- Utilizzato per effettuare **output** di elementi sulla console
- La parola chiave è `print`

```
x = 1
print(x)
x_str = str(x)
print("my fav num is", x, ".", "x =", x)
print("my fav num is " + x_str + ". " + "x = " + x_str)
```



INPUT/OUTPUT: `input("")`

- Il comando **`input`** effettua la stampa di qualunque cosa ci sia tra doppi apici e attende che l'utente inserisca qualcosa da tastiera
- Una volta che digita INVIO, il *bind* avviene tra il valore inserito da tastiera e la variabile che viene assegnata tramite **`input`**

```
text = input("Type anything... ")  
  
print(5*text)
```

- **`input`** **restituisce una stringa**, quindi è necessario il cast se dobbiamo operare per esempio con numeri

```
num = int(input("Type a number... "))  
  
print(5*num)
```




Operatori di comparazione tra int, float, string

- Assumiamo `i` e `j` come nomi di variabile
- Le comparazioni di seguito restituiscono un booleano (True, False)

`i > j`

`i >= j`

`i < j`

`i <= j`

`i == j` → **equality** test, True se il valore di `i` è lo stesso di `j`

`i != j` → **inequality** test, True se il valore di `i` non è lo stesso di `j`



Operatori logici tra booleani

- Assumiamo a e b nomi di variabili booleane

not a $\rightarrow$ True se a è False
False se a è True

a and b $\rightarrow$ True se entrambi sono True

a or b $\rightarrow$ True se una delle due è True

A	B	A and B	A or B
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False



Flusso di controllo - Branching

```
if <condition>:  
    <expression>  
    <expression>  
    ...
```

```
if <condition>:  
    <expression>  
    <expression>  
    ...  
else:  
    <expression>  
    <expression>  
    ...
```

```
if <condition>:  
    <expression>  
    <expression>  
    ...  
elif <condition>:  
    <expression>  
    <expression>  
    ...  
else:  
    <expression>  
    <expression>  
    ...
```

- <condition> può essere True o False
- Si valuta l'espressione nel blocco se <condition> è True



Indentazione

- L'indentazione in Python conta!
- Viene utilizzata per definire un **blocco di istruzioni**

```
x = float(input("Enter a number for x: "))
y = float(input("Enter a number for y: "))
if x == y:
    print("x and y are equal")
    if y != 0:
        print("therefore, x / y is", x/y)
elif x < y:
    print("x is smaller")
else:
    print("y is smaller")
print("thanks!")
```



Operatore = vs Operatore ==

```
x = float(input("Enter a number for x: "))
y = float(input("Enter a number for y: "))
if x == y:
    print("x and y are equal")
    if y != 0:
        print("therefore, x / y is", x/y)
elif x < y:
    print("x is smaller")
else:
    print("y is smaller")
print("thanks!")
```

What if x = y here?
get a `SyntaxError`



Flusso di controllo: ciclo while

```
while <condition>:  
    <expression>  
    <expression>  
    ...
```

1. <condition> viene valutata come un booleano
2. se <condition> è True, allora esegui tutte le istruzioni nel blocco while
3. controlla <condition>
4. ripeti finché <condition> è False



Flusso di controllo: ciclo for

```
for <variable> in range(<some_num>) :  
    <expression>  
    <expression>  
    ...
```

- Ogni volta che siamo nel ciclo, <variable> assume un valore
- Prima volta, <variable> l'estremo inferiore del set specificato in range
- Le successive volte, <variable> assume il valore precedente sommato di 1
- E così via...



Flusso di controllo: ciclo for

`range(start,stop,step)`

- I valori di default sono `start = 0` e `step = 1`, e sono opzionali
- Il ciclo esegue fino a che il valore diventa `stop - 1`
- E.g.:

```
mysum = 0
for i in range(7, 10):
    mysum += i
print(mysum)
```

```
mysum = 0
for i in range(5, 11, 2):
    mysum += i
print(mysum)
```




Istruzione break

- Esce immediatamente qualunque sia il tipo e lo stato corrente del ciclo utilizzato
- Salta la valutazione delle espressioni rimanenti nel blocco di codice
- Esce soltanto nel contesto del ciclo più interno!

```
while <condition_1>:  
    while <condition_2>:  
        <expression_a>  
        break  
        <expression_b>  
    <expression_c>
```



Istruzione break: esempio

```
mysum = 0
for i in range(5, 11, 2):
    mysum += i
    if mysum == 5:
        break
    mysum += 1
print(mysum)
```

- Cosa succede a questo programma?



Stringhe e Cicli

- Gli snippet di codice seguenti fanno la stessa cosa
- Quello più in basso è più “pythonic”

```
s = "abcdefgh"
```

```
for index in range(len(s)):
    if s[index] == 'i' or s[index] == 'u':
        print("There is an i or u")
```

```
for char in s:
    if char == 'i' or char == 'u':
        print("There is an i or u")
```

Stringhe e Cicli:

Esempio ROBOT CHEERLEADERS



```
an_letters = "aefhilmnorsxAEFHILMNORSX"
```

```
word = input("I will cheer for you! Enter a word: ")
times = int(input("Enthusiasm level (1-10): "))
```

```
i = 0
while i < len(word):
    char = word[i]
    if char in an_letters:
        print("Give me an " + char + "! " + char)
    else:
        print("Give me a  " + char + "! " + char)
```

```
for char in word:
```

```
    i += 1
print("What does that spell?")
for i in range(times):
    print(word, "!!!")
```



Stringhe e Cicli: Esempio 2

```
s1 = "unina u rock"
s2 = "i rule unina"
if len(s1) == len(s2):
    for char1 in s1:
        for char2 in s2:
            if char1 == char2:
                print("common letter")
                break
```



Funzioni in Python

- Le **funzioni** permettono di scrivere codice riusabile
- Le funzioni non vengono eseguite nel programma finché non vengono **chiamate** o **invocate**
- Caratteristiche di una funzione:
 - Ha un **nome**
 - Ha dei **parametri** (0 or più)
 - Ha una **docstring** (opzionale ma raccomandata)
 - La prima istruzione (commento) di una funzione che funge da documentazione
 - Ha un **corpo**
 - **Ritorna** qualcosa



Come scrivere e invocare una funzione in Python

```
def is_even( i ) :  
    """  
    Input: i, a positive int  
    Returns True if i is even, otherwise False  
    """  
    print("inside is_even")  
    return i%2 == 0  
  
is_even(3)
```

keyword

name

parameters or arguments

specification, docstring

body

later in the code, you call the function using its name and values for parameters



Corpo della funzione

```
def is_even( i ):  
    """  
    Input: i, a positive int  
    Returns True if i is even, otherwise False  
    """
```

```
    print("inside is_even")
```

```
    return i%2 == 0
```

keyword

expression to
evaluate and return

run some
commands



Visibilità (scope) di una variabile

- **formal parameter (parametri formali)** sono legati al valore di un **actual parameter (parametro effettivo)** quando una funzione viene invocata
- Viene creato un nuovo **scope/frame/environment** quando si entra in una funzione
- **scope** è il mapping tra i nomi e gli oggetti

```
def f ( x ) :  
    x = x + 1  
    print('in f(x): x =', x)  
    return x
```

*formal
parameter*

*Function
definition*

```
x = 3  
z = f ( x )
```

*actual
parameter*

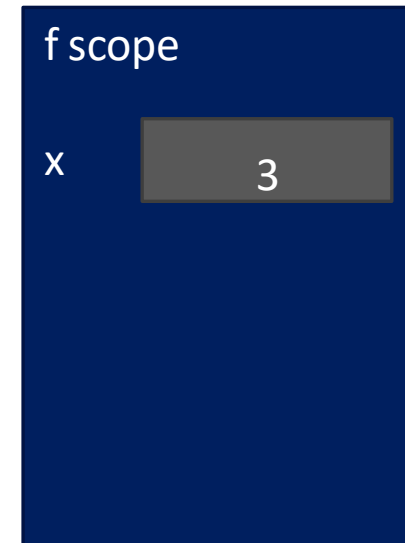
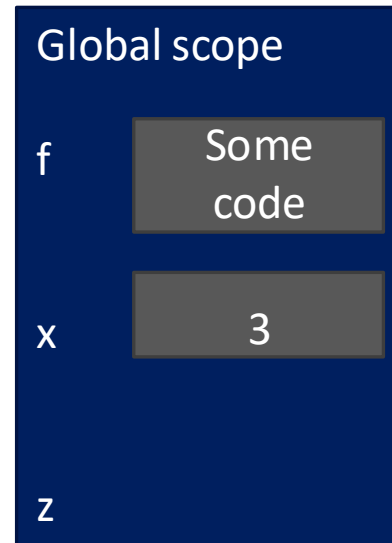
Main program code
* initializes a variable x
* makes a function call f(x)
* assigns return of function to variable z



Visibilità (scope) di una variabile

```
def f( x ) :  
    x = x + 1  
    print('in f(x): x =', x)  
    return x
```

```
x = 3  
z = f( x )
```

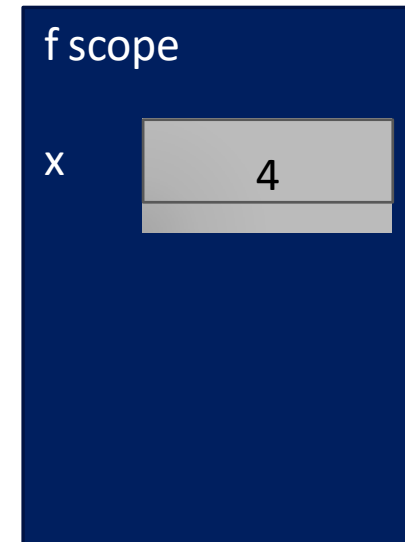
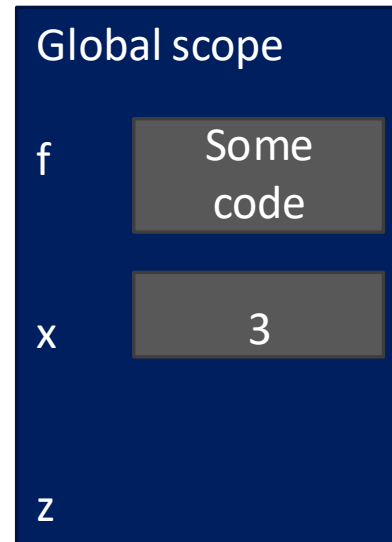




Visibilità (scope) di una variabile

```
def f( x ) :  
    x = x + 1  
    print('in f(x): x =', x)  
    return x
```

```
x = 3  
z = f( x )
```

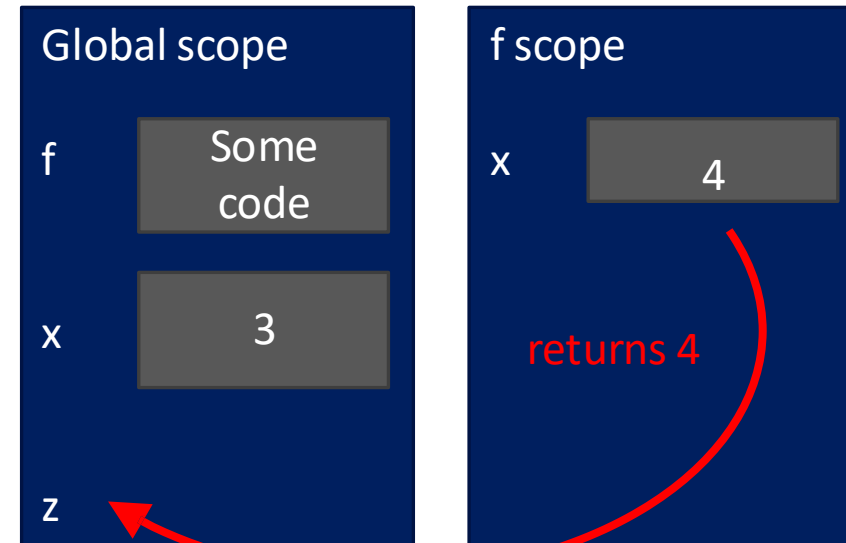




Visibilità (scope) di una variabile

```
def f( x ) :  
    x = x + 1  
    print('in f(x): x =', x)  
    return x
```

```
x = 3  
z = f( x )
```

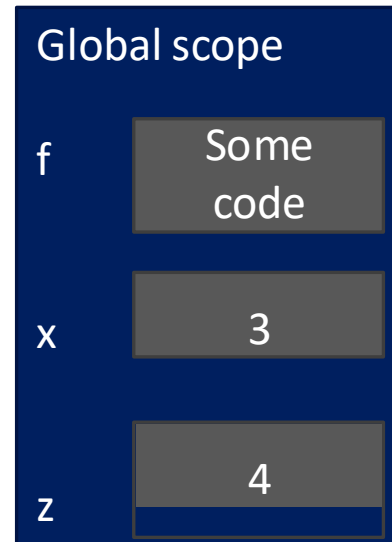




Visibilità (scope) di una variabile

```
def f( x ) :  
    x = x + 1  
    print('in f(x): x =', x)  
    return x
```

```
x = 3  
z = f( x )
```





Istruzione return e il tipo None

```
def is_even( i ) :  
    """  
    Input: i, a positive int  
    Does not return anything  
    """
```

```
i%2 == 0
```

without a return
statement

- Python ritorna il valore **None**, se non si usa l'istruzione **return** !
- None rappresenta l'assenza di valore



Funzioni come argomenti

Gli argomenti di funzione possono essere di qualunque tipo, anche funzioni!

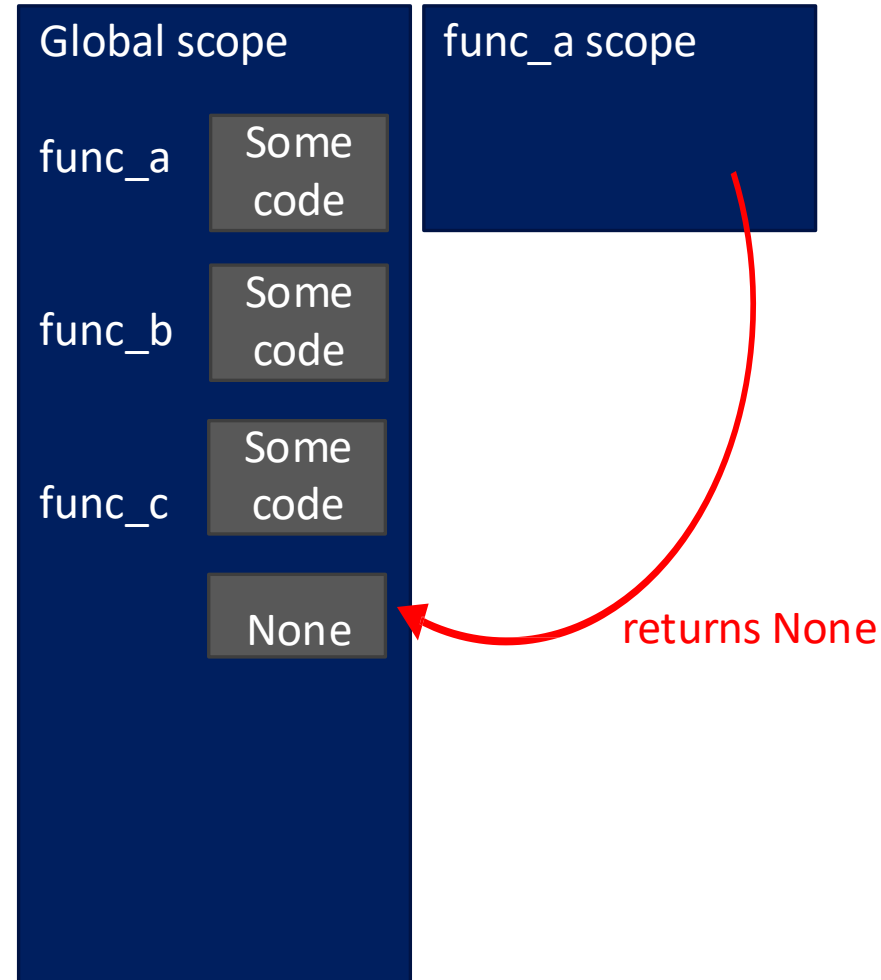
```
def func_a():  
    print 'inside func_a'  
  
def func_b(y):  
    print 'inside func_b'  
    return y  
  
def func_c(z):  
    print 'inside func_c'  
    return z()  
  
print func_a()  
print 5 + func_b(2)  
print func_c(func_a)
```

call func_a, takes no parameters
call func_b, takes one parameter
call func_c, takes one parameter, another function



Funzioni come argomenti

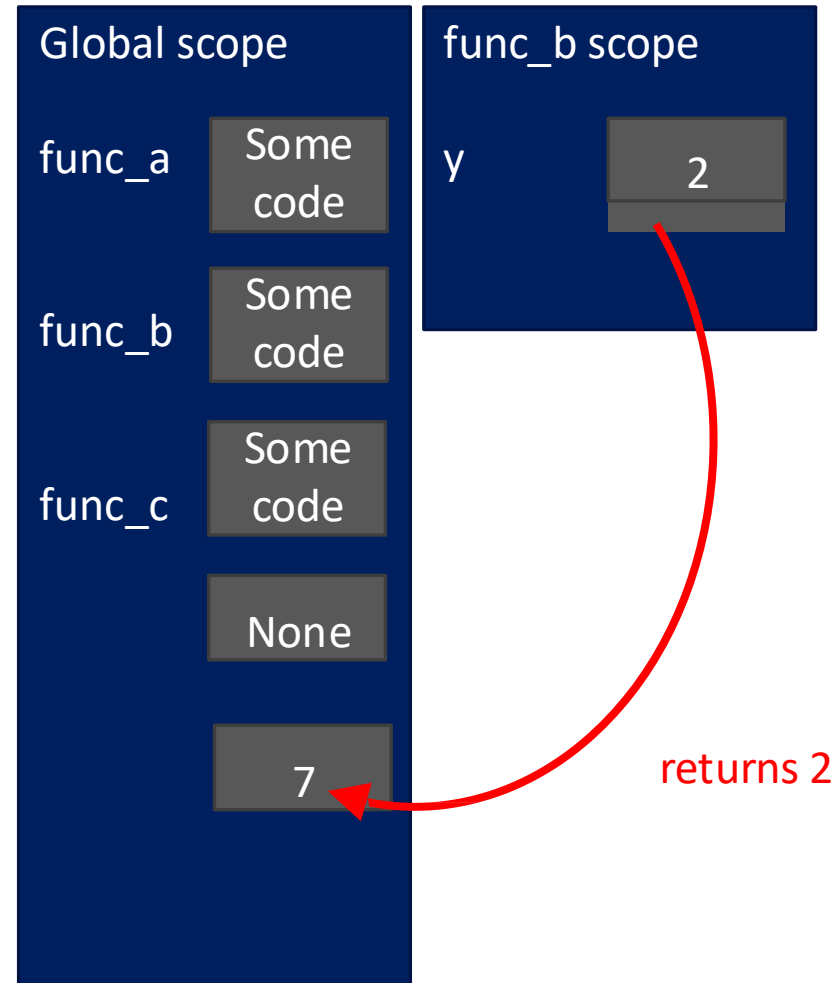
```
def func_a():  
    print 'inside func_a'  
def func_b(y):  
    print 'inside func_b'  
    return y  
def func_c(z):  
    print 'inside func_c'  
    return z()  
  
print func_a()  
print 5 + func_b(2)  
print func_c(func_a)
```





Funzioni come argomenti

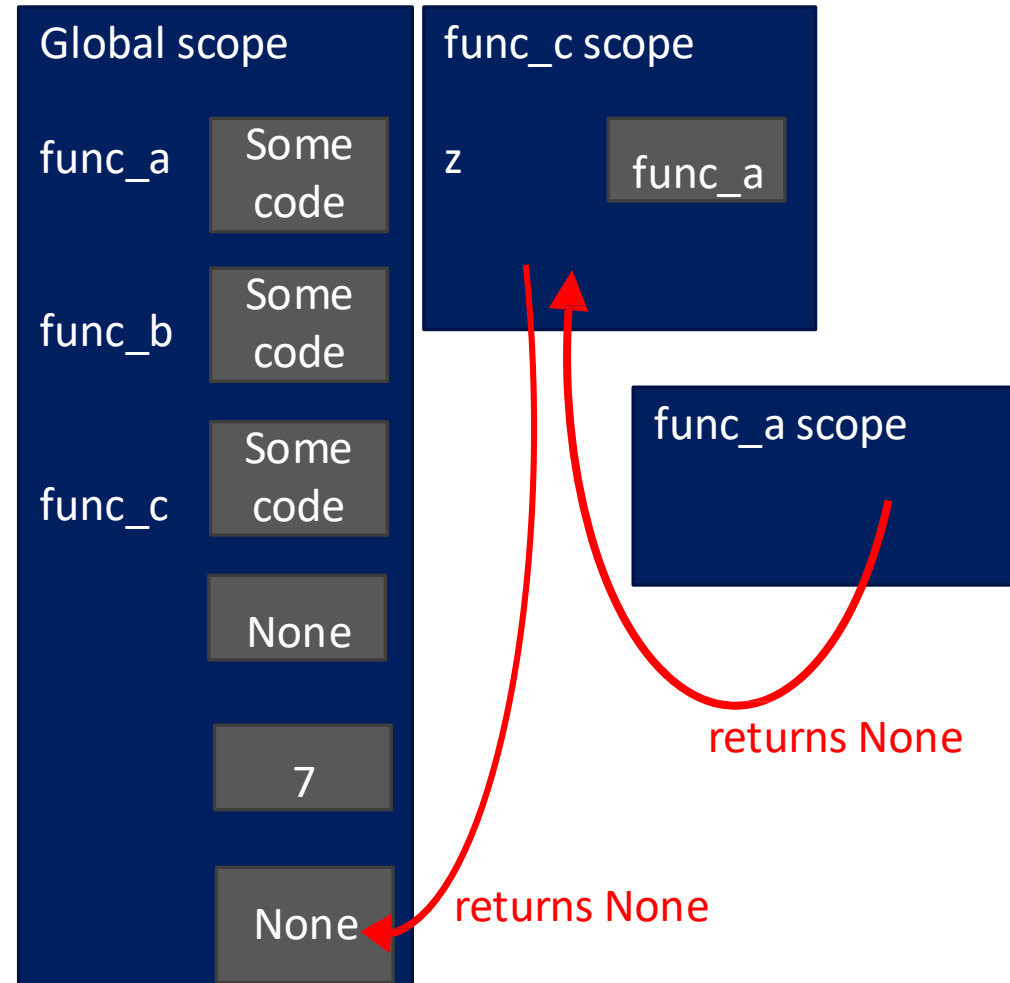
```
def func_a():  
    print 'inside func_a'  
def func_b(y):  
    print 'inside func_b'  
    return y  
def func_c(z):  
    print 'inside func_c'  
  
print func_a()  
print 5 + func_b(2)  
print func_c(func_a)
```





Funzioni come argomenti

```
def func_a():  
    print 'inside func_a'  
def func_b(y):  
    print 'inside func_b'  
    return y  
def func_c(z):  
    print 'inside func_c'  
    return z()  
print func_a()  
print 5 + func_b(2)  
print func_c(func_a)
```





Esempio di scope

- Dentro una funzione, **possiamo accedere** ad una variabile definita al di fuori della funzione
- Dentro una funzione, **non possiamo modificare** una variabile definita al di fuori della funzione – possiamo usare **variabili globali**, ma non è una buona pratica

```
def f(y):  
    x = 1  
    x += 1  
    print(x)
```

*x is re-defined
in scope of f*

```
x = 5  
f(x)  
print(x)
```

*different x
objects*

```
def g(y):  
    print(x)  
    print(x + 1)
```

*x from
outside g*

```
x = 5  
g(x)  
print(x)
```

*x inside g is picked up
from scope that called
function g*

```
def h(y):  
    x += 1
```

```
x = 5  
h(x)  
print(x)
```

*UnboundLocalError: local variable
'x' referenced before assignment*



Esempio di scope

- Dentro una funzione, **possiamo accedere** ad una variabile definita al di fuori della funzione
- Dentro una funzione, **non possiamo modificare** una variabile definita al di fuori della funzione – possiamo usare **variabili globali**, ma non è una buona pratica

<pre>def f(y): x = 1 x += 1 print(x) x = 5 f(x) print(x)</pre>	<pre>def g(y): print(x) print(x + 1) x = 5 g(x) print(x)</pre>	<pre>def h(y): x += 1 x = 5 h(x) print(x)</pre>
---	---	--

x from global/main program scope



Esempio complesso sullo scope

IMPORTANT
and TRICKY!

Python Tutor è il tuo miglior amico!

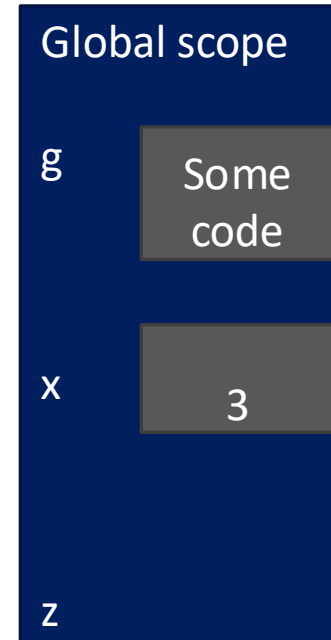
<http://www.pythontutor.com/>



Esempio complesso sullo scope

```
def g(x):  
    def h():  
        x = 'abc'  
    x = x + 1  
    print('g: x =', x)  
    h()  
    return x
```

Some code



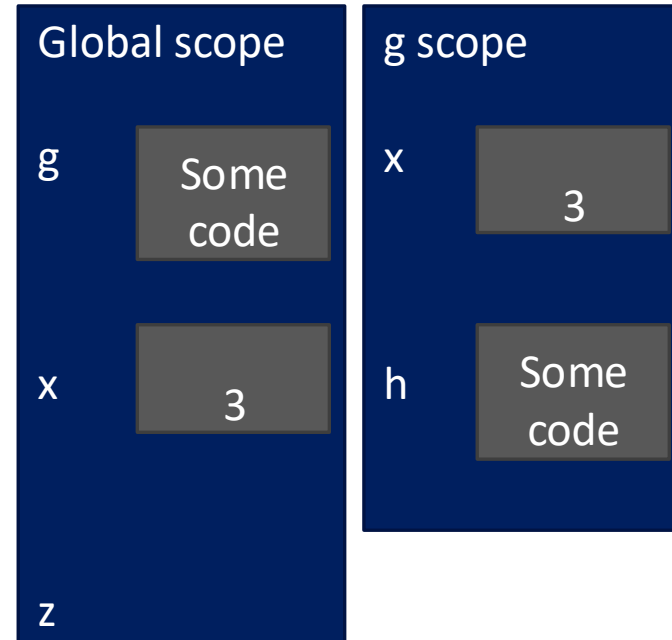
```
x = 3  
z = g(x)
```



Esempio complesso sullo scope

```
def g(x):  
    def h():  
        x = 'abc'  
    x = x + 1  
    print('g: x =', x)  
    h()  
    return x
```

```
x = 3  
z = g(x)
```

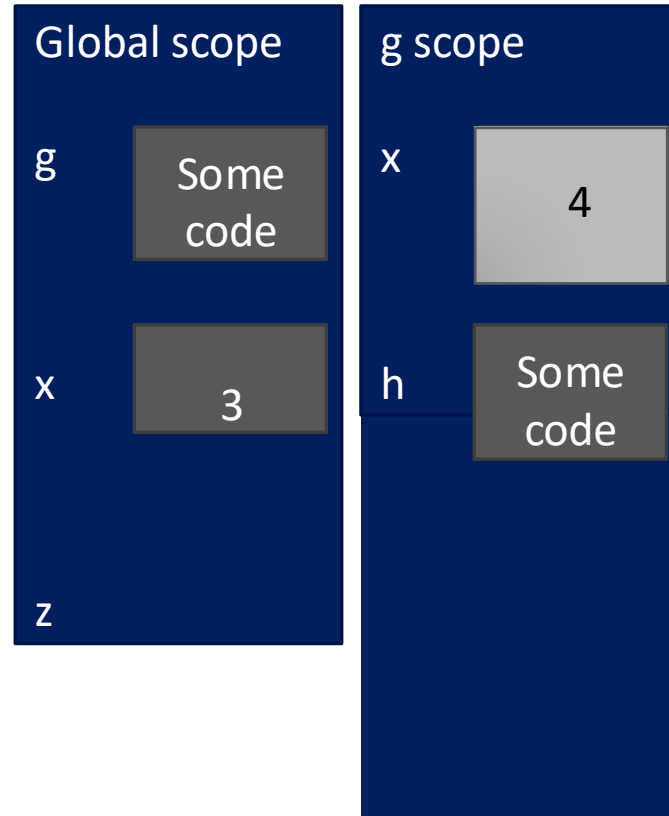




Esempio complesso sullo scope

```
def g(x):  
    def h():  
        x = 'abc'  
    x = x + 1  
    print('g: x =', x)  
    h()  
    return x
```

```
x = 3  
z = g(x)
```

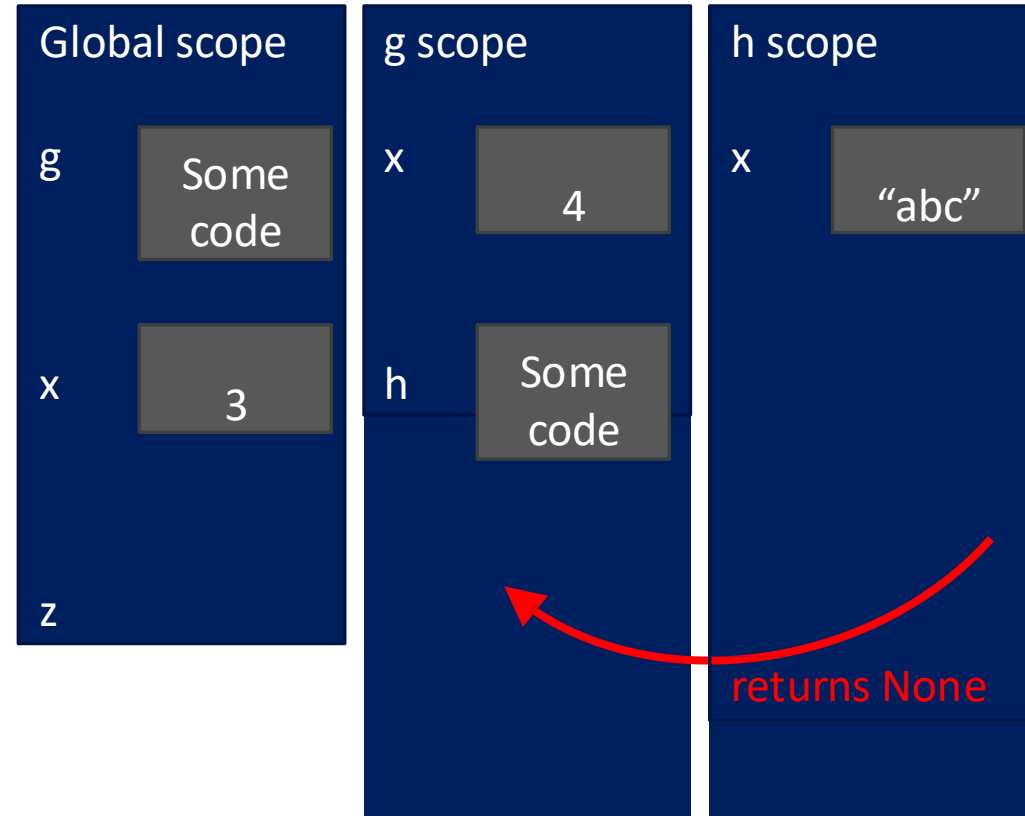




Esempio complesso sullo scope

```
def g(x):  
    def h():  
        x = 'abc'  
    x = x + 1  
    print('g: x =', x)  
    h()  
    return x
```

```
x = 3  
z = g(x)
```

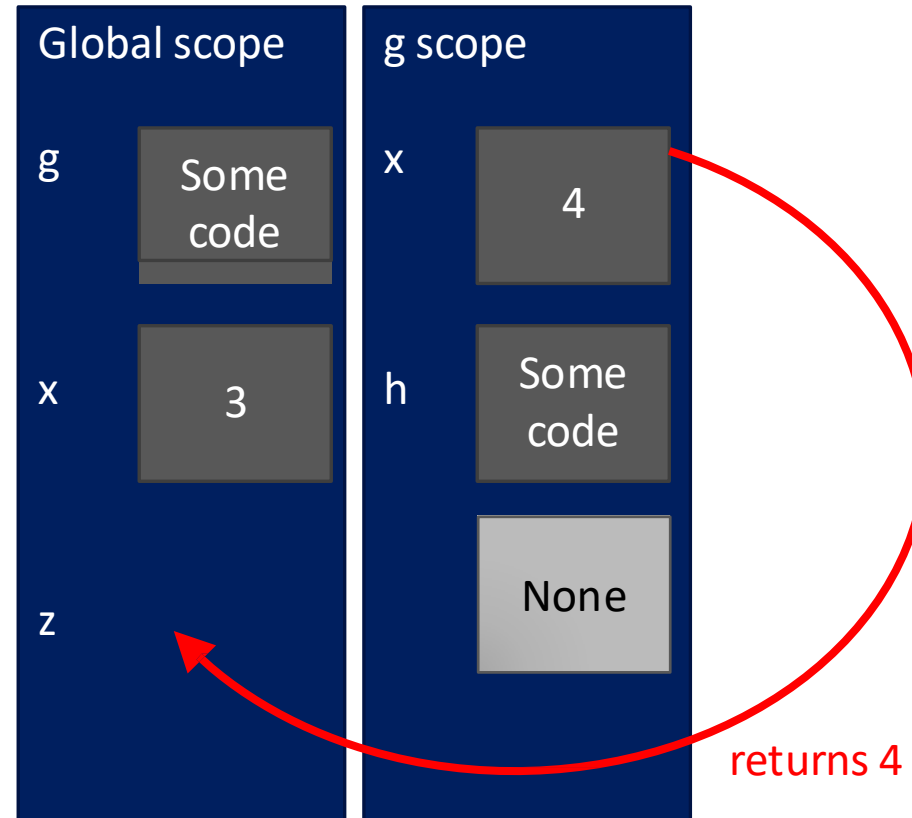




Esempio complesso sullo scope

```
def g(x):  
    def h():  
        x = 'abc'  
    x = x + 1  
    print('g: x =', x)  
    h()  
    return x
```

```
x = 3  
z = g(x)
```

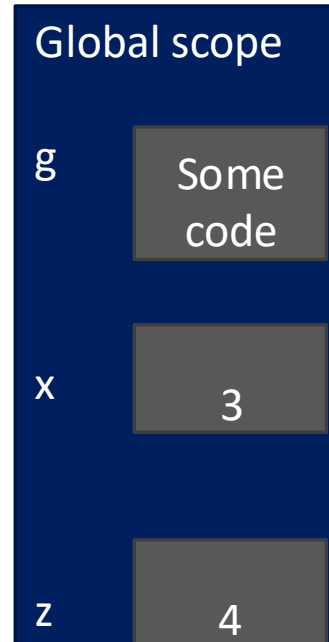




Esempio complesso sullo *scope*

```
def g(x):  
    def h():  
        x = 'abc'  
    x = x + 1  
    print('g: x =', x)  
    h()  
    return x
```

```
x = 3  
z = g(x)
```





La funzione *main* ()

- Generalmente la funzione **main ()** è il punto di ingresso principale di un programma
- Tuttavia l'interprete Python **esegue il codice fin dalla prima riga di un codice sorgente**, a prescindere dal fatto che ci sia o meno la definizione della funzione `main ()`
- **In Python non esiste una funzione main()**
 - Quando viene dato all'interprete il comando di esecuzione di un programma Python, viene eseguito il codice che si trova al livello 0 di indentazione
- Tuttavia vengono definite alcune variabili speciali come **`__name__`**
- **`__name__`** è una variabile *built-in* che valuta il nome del modulo corrente
 - Se il file **.py** viene **eseguito come programma principale**, l'interprete imposta la variabile **`__name__`** in modo che abbia il valore **`__main__`**
 - Se il file **.py** viene **importato da un altro modulo**, **`__name__`** sarà impostato sul **nome del modulo**.



Esempio di `main()` \1

```
# Python program to demonstrate main() function
```

```
print("Hello")
```

```
# Defining main function
```

```
def main():
```

```
    print("hey there")
```

```
# Using the special variable
```

```
# __name__
```

```
if __name__=="__main__":
```

```
    main()
```

• Output

Hello

hey there



Esempio di `main()` \2

```
# File1.py

print("File1 __name__ = %s" %__name__)

if __name__ == "__main__":
    print("File1 is being run directly")
else:
    print("File1 is being imported")
```

Output:

```
File1 __name__ = __main__
File1 is being run directly
```

```
# File2.py

import File1

print("File2 __name__ = %s" %__name__)

if __name__ == "__main__":
    print("File2 is being run directly")
else:
    print("File2 is being imported")
```

Output:

```
File1 __name__ = File1
File1 is being imported
File2 __name__ = __main__
File2 is being run directly
```